

M36 Development Directions for StratLake Trade Engine

Executive summary

Using the enabled connector set, I found only one enabled connector and used only the specified repository: **GitHub**, specifically `christophermoverton/stratlake-trade-engine`. The repository is a public Python-based quantitative research platform with 391 commits visible on the repository page at inspection time, and the README defines it as a **deterministic research platform** for curated market data, backtests, validation, and auditable artifacts—explicitly **not** a live-trading or ingestion system. The repo page also showed **0 open pull requests** at the time of inspection. ¹

The most important finding is that I did **not** find a literal, explicit technical artifact named “**M36**” in the repository materials I reviewed. The latest explicit milestone in the repo’s own narrative is **M35**, which is documented as “**Artifact Lineage, Discovery, and Robustness-Aware Research Catalog**.” However, the fetched GitHub issue objects for M35 work items show a numeric `milestone: 36` field even though their titles and bodies clearly refer to **M35**. That makes “M36” most plausibly either a **GitHub milestone identifier / ordinal** attached to M35 work, or a **user shorthand for the next milestone after M35**, rather than an implemented code module, protocol, or model version already present in the repo. ² ³ ⁴

⁵

The repo is already strong on **artifact-first reproducibility**, notebook/CLI parity, deterministic validation, governance observability, and robustness-aware research review. The M35 layer adds read-only evidence cataloging, evidence-oriented query filters, lineage edges, a local evidence explorer, and notebook helpers under `src.catalog`, with focused regression tests and cross-platform CI/release workflows. ⁶ ² ³

⁷

⁸

⁹

¹⁰

The highest-value **M36** direction is therefore **not** “add live trading immediately.” The evidence argues for a successor milestone centered on **scale, interoperability, release correctness, and security hardening**: an optional derived metadata index for faster catalog search, standards-based lineage export such as OpenLineage and/or W3C PROV, stronger dataset/feature lineage contracts, release/version alignment, and CI/CD hardening. Those directions align the repo with external best practices seen in MLflow, DVC, OpenLineage, Qlib, LEAN, and NautilusTrader, while preserving StratLake’s current artifact-first, deterministic, research-review posture. ¹¹

Scope and method

The only enabled connector available for this task was **GitHub**, and I constrained repository inspection to `christophermoverton/stratlake-trade-engine`, per your instruction. I then used external web sources only for public official documentation, competitor repositories, and original or primary academic material where that materially improved the analysis. The target platform, team size, and budget remain

unspecified, so recommendations below are framed as strategic options with effort levels rather than staffing-specific plans.

Within the repository, I reviewed the README and M35 documentation, code entry points under `src.catalog` and `src.cli`, the release and CI workflows, package metadata, and the focused M35 validation tests. Externally, I used official or primary sources for MLflow tracking, DVC documentation, OpenLineage, W3C PROV, Qlib documentation, and the public GitHub repositories for QuantConnect LEAN and NautilusTrader, as well as a recent provenance paper relevant to ML lineage. 12 13 14 15 16 2
3 7 8 9 10 17

Repository findings relevant to any future M36

The repo's present architecture is coherent and milestone-driven. The README describes StratLake as a deterministic research platform, not a live-trading engine, and the latest public milestone narrative on the repo side runs from M23 through M35. Several milestone layers are especially relevant to any next-step "M36" thinking: M29 introduced a read-only research catalog and lineage explorer; M30 added statistical diagnostics and readiness artifacts; M31 routed readiness into promotion policy; M32 added promotion-governance observability; M33 added cross-platform validation and packaging/release automation posture; M34 added statistical robustness and overfitting guardrails; and M35 extended catalog/query/lineage surfaces so robustness, governance, milestone-validation, and release-validation outputs become first-class review evidence. 18 2 3

At the code level, the relevant current M35 surface is concentrated under `src.catalog` and `src.cli`. The public package exports show catalog building, evidence explorer rendering, lineage generation, notebook evidence helpers, query functions, and validation helpers from a shared package surface. The query CLI exposes filters for evidence-oriented fields such as `record_family`, `robustness_status`, `wfe_status`, `sample_size_status`, `trade_count_status`, `sensitivity_status`, `fragility_status`, `multiple_testing_status`, `temporal_validation_status`, `governance_status`, `promotion_review_status`, and readiness-presence booleans. The explorer CLI adds read-only rendering of selected runs or filtered evidence into Markdown, JSON, or table output. 6 8 9

The schema work is explicit. `CatalogRecord` now includes evidence fields such as `record_family`, `robustness_status`, `wfe_status`, `sample_size_status`, `trade_count_status`, `sensitivity_status`, `fragility_status`, `multiple_testing_status`, `temporal_validation_status`, `governance_status`, `promotion_review_status`, `validation_readiness_present`, and `release_validation_present`. The foundation doc maps those fields to source-of-truth artifacts like `robustness_summary.json`, `walk_forward_efficiency.csv`, `sample_size_validation.json`, `sensitivity_summary.csv`, `multiple_testing_summary.json`, `leakage_validation.json`, governance summaries, milestone validation summaries, and release-validation artifacts. 19 3

The M35 work was tracked through a clear issue sequence. Issue #401 describes M35 as complete and lists fulfilled implementation issues #394 through #400: evidence catalog record families and indexing, robustness/governance query discovery, evidence lineage, local evidence explorer, notebook/API helpers, deterministic validation coverage, and release docs / checklist. It also records that the branch CI was green

across Ubuntu, Windows, and macOS smoke checks, ruff, docs path lint, deterministic rerun, packaging build, and pytest. 5

Tests are not superficial. The dedicated M35 validation file asserts that the evidence-discovery chain is deterministic across repeated catalog builds and renders; that discovery tools do not mutate source artifacts; that outputs avoid absolute paths and file:// URIs; that empty and sparse artifact roots are safe; and that governance evidence remains observable and read-only rather than mutating promotion status. The validation guide also documents targeted pytest slices, a CI-safe example script, and broader regressions over catalog/robustness/governance/lineage/explorer/notebook surfaces. 7 10

The current CI/CD posture is solid for a research repository. ci.yml runs a cross-platform smoke matrix on Ubuntu, Windows, and macOS; uses cached Python setup; installs with the committed requirements-dev.lock; runs focused ruff and pytest slices; runs docs/path lint; runs deterministic rerun validation; validates packaging; and runs the full pytest suite. release.yml is tag-driven on v*, performs constrained installs and a release pytest slice, runs docs/path lint, builds wheel and sdist, renders deterministic release notes, uploads artifacts, and creates a GitHub Release. milestone_validation.yml is separate and runs the milestone validation bundle. Packaging metadata is declared in pyproject.toml, and the dependency lock is committed in requirements-dev.lock. 13 14 15 16 20

What M36 most likely means

The repository evidence supports a narrow set of plausible meanings for “M36,” and it does **not** support several others.

Plausible meaning of “M36”	Evidence for	Evidence against	Assessment
GitHub issue milestone identifier / ordinal	Fetches issue objects for M35 issues #394 and #401 show milestone: 36 even though their titles and bodies explicitly label the work as M35. 4 5	The issue content itself never describes a technical milestone named “M36.” 4	High confidence as an explanation for the number “36,” but low technical significance by itself.
The next StratLake milestone after M35	The repo’s own externally visible milestone narrative reaches M35, and M35 was just merged and release-prepared. That makes a successor milestone interpretation natural. 2 5	I did not find a direct M36 title/body in the reviewed repo artifacts.	High confidence as the most useful product-planning interpretation.

Plausible meaning of “M36”	Evidence for	Evidence against	Assessment
A code module, protocol version, or model version named M36	No affirmative repository evidence found.	The M35 code/docs are catalog, governance, robustness, and exploration focused; no module/exported symbol/ documented interface reviewed used “M36” as a technical object. ⁶ ² ³	Low confidence; treat as unsupported unless deeper repository evidence appears.

The practical implication is that development planning should treat “M36” as **the next milestone direction after M35**, while explicitly noting that the repo itself does not yet define that milestone in the reviewed materials. The safest analytical move is therefore to derive M36 from the repo’s current strengths and constraints rather than from a nonexistent literal “M36” spec. ² ³ ⁵

External landscape and comparison

Recent and current external practice points in a consistent direction. MLflow’s official tracking documentation now emphasizes logging **parameters, code versions, metrics, and output files**, supports dataset-aware search and direct model IDs, and distinguishes a **backend store** for metadata from an **artifact store** for run outputs. OpenLineage defines an open lineage model around **jobs, runs, and datasets** with extensible facets and broad integrations. DVC’s documentation centers on **data pipelines, experiment management, versioning data and models, and data registries**. Together, these tools indicate that artifact-first systems increasingly benefit from optional metadata stores, stronger search, and standardized lineage rather than ad hoc local-only cataloging. ²¹

Academic work is also moving in that direction. W3C PROV remains the foundational provenance vocabulary around entities, activities, and agents, with explicit goals around reproducibility, versioning, and derivation. A 2025 provenance paper, **yProv4ML**, argues that common ML tooling such as MLflow captures key run information but pays relatively limited attention to lineage and proposes PROV-JSON capture with minimal code changes. Meanwhile, recent financial-ML work continues to treat backtest overfitting as an active practical problem, reinforcing the value of StratLake’s M34/M35 robustness-and-review emphasis rather than weakening it. ²²

Competitor projects show two slightly different “north stars.” LEAN and NautilusTrader optimize for **research-to-live execution systems** with event-driven architectures, multi-market or multi-venue support, and operational deployment surfaces. Qlib, by contrast, is more research- and workflow-centric, with experiment management, portfolio backtesting, and online-serving components. StratLake today sits closer to the **research-governance / artifact reproducibility** pole than to the **live execution** pole. That is not a weakness by itself; it simply means an M36 that tried to compete directly with LEAN or NautilusTrader on live execution would be a strategic jump, not a natural adjacent step. ²³

System	Core orientation	Relevant strengths	Comparison to current StratLake
StratLake current repo	Deterministic, artifact-first research and review platform	Strong reproducibility, notebook/CLI parity, governance observability, robustness-aware evidence catalog, cross-platform CI. ¹⁸ ²	Strong on reviewability and deterministic artifacts; weaker on scalable search, external lineage standards, and production integration.
MLflow	Experiment tracking / model management	Logs params, code versions, metrics, and output files; supports backend and artifact stores; adds dataset-aware model search and model-dataset links. ²⁴	Best-fit comparator for M36 catalog/search ergonomics and metadata storage.
DVC	Data/versioning and experiment workflows	Pipelines, experiment tracking, versioning data/models, registry patterns. ²⁵	Best-fit comparator for stronger feature-dataset version contracts in StratLake.
OpenLineage + W3C PROV	Interoperable lineage / provenance standards	Open model for jobs-runs-datasets with extensible facets; PROV standardizes provenance semantics around entities, activities, and agents. ²⁶	Best-fit comparator for making StratLake evidence exportable beyond internal schemas.
Qlib	Quant research workflow platform	Workflow management, backtesting, experiment recorder, online serving, point-in-time data support. ²⁷	Broader integrated quant-research surface than StratLake in experiment management and serving.
LEAN	Open-source algorithmic trading engine	Event-driven, modular, live-trading support, CLI, large ecosystem footprint. ²⁸	Competes on execution breadth, not on evidence catalog / governance review depth.
NautilusTrader	Production-grade research/live trading engine	Deterministic event-driven research/live parity, multi-venue adapters, Docker, nanosecond-resolution backtesting. ²⁹	Strongest warning against accidental half-step live aspirations; if StratLake ever goes there, the bar is high.

Gaps, technical debt, and risks

The largest **functional gap** is not inside M35 itself but around **scale and interoperability**. M35 is deliberately read-only, local, and non-persistent: no second registry, no database, no cache, no search

backend, no graph store, no dashboard service. That is an intentional architectural boundary and a sound one for milestone scope. But compared with MLflow, DVC, and OpenLineage ecosystems, it means the current catalog layer is likely to become less ergonomic as artifact history, team size, or cross-tool integration needs grow. That is an inference from the repo's explicit non-goals combined with external platform designs. ² ³ ²¹

There is also a concrete **release/versioning debt**: `pyproject.toml` declares project version `0.1.0`, while Issue #401 proposes the release tag candidate `v0.35.0-artifact-lineage-research-catalog`. That mismatch creates ambiguity for users, package consumers, release notes, and provenance records. If the repo is milestone-versioned in human communication but package-versioned separately in metadata, that should be formalized; otherwise, it should be aligned. ¹⁶ ⁵

A second concrete risk is **workflow-trigger drift**. Issue #401 names the M35 branch as `feature/m35-artifact-lineage-research-catalog`, while `milestone_validation.yml` only auto-triggers on pushes to `milestone/**` and `m22/**`, and on pull requests whose `head_ref` starts with those prefixes. That means the dedicated milestone-validation workflow can miss exactly the kind of `feature/m35-*` branch used by the repo's recent milestone process unless a manual dispatch or different path is used. ¹⁵ ⁵

The repo also carries a clear **CI/CD supply-chain hardening gap**. The README explicitly notes that workflows use maintained major versions such as `actions/checkout@v4` and `actions/setup-python@v5`, while full SHA pinning is deferred. GitHub's own security guidance says pinning actions to a full-length commit SHA is the only immutable-release approach and recommends it as a mitigation against compromised upstream actions. This is a prudent "M36 hardening" candidate precisely because the repo already has strong deterministic validation and is close to production-grade release hygiene. ¹² ¹³ ¹⁴ ³⁰

Strategically, there is a **boundary risk** around live execution. StratLake states plainly that it is for research review and not live trading, whereas LEAN and NautilusTrader lean heavily into research-to-live parity. If M36 is not explicit about whether StratLake remains a research-review platform or begins a narrow handoff toward execution, the roadmap could blur product identity and produce expensive half-steps. In other words, the biggest strategic risk is not under-building; it is **ambiguous building**. ³¹

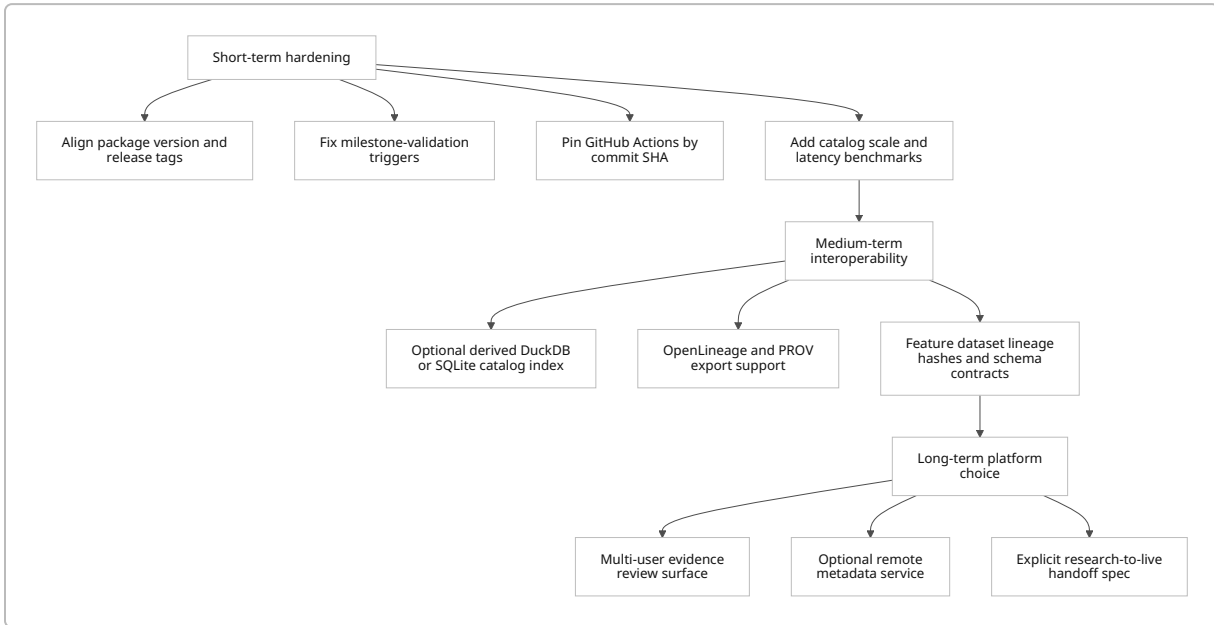
Recommended M36 development directions

My recommendation is to define **M36** as a successor milestone with the following theme:

"Scalable Evidence Interoperability and Release Hardening."

That theme fits the repo's existing trajectory better than a jump to live trading. It extends M29-M35 naturally: from reproducible artifacts, to diagnostics and governance, to robustness-aware cataloging, and then to **faster search, stronger provenance contracts, interoperable lineage, and corrected release/security posture**. It also preserves the repo's artifact-first design by favoring **optional derived stores and exporters** over canonical databases or opaque services. ² ³ ³²

Proposed roadmap



Prioritized development directions

Horizon	Direction	Effort	Key milestones	Main risks
Short term	Release and CI coherence	Low to medium	Align versioning semantics; expand milestone-validation triggers; pin actions to SHAs; document release policy clearly	Low implementation risk; high payoff in trust and maintainability
Short term	Catalog scale baselines	Medium	Add benchmark fixtures, scan timing, memory snapshots, regression thresholds	Reveals performance pain that may require deeper refactors
Medium term	Optional derived metadata index	Medium	Add local DuckDB/SQLite export or cache for catalog/query acceleration while preserving source artifacts as truth	Risk of accidental “second source of truth” unless explicitly marked derived
Medium term	Standards-based lineage export	Medium	Emit OpenLineage events and/or W3C PROV bundles from existing catalog/lineage records	Mapping internal semantics cleanly to standards may need schema decisions

Horizon	Direction	Effort	Key milestones	Main risks
Medium term	Stronger dataset and feature lineage	High	Add dataset fingerprints, feature-manifest hashes, source dataset references, contract validation	Can touch many artifact producers and raise migration complexity
Long term	Collaborative review surface	High	Team-oriented explorer/UI over the same read-only contracts	Scope creep into service/database operations
Long term	Explicit handoff to execution layer	High / strategic	Either formalize “research-only forever” or define a narrow export/handoff contract to external execution engines	Strategic drift if not tightly bounded

Recommended first five actionable tasks

1. **Resolve version semantics immediately.** Decide whether human-facing milestones and package versions should stay separate. If separate, document the policy and surface both in artifacts. If not, align `pyproject.toml` with release tags before the next release. [16](#) [5](#)
2. **Update `milestone_validation.yml` triggers to cover the actual branch naming convention.** At minimum, include `feature/m*` or use a reusable workflow callable from CI so milestone validation cannot silently miss milestone branches. [15](#) [5](#)
3. **Pin all third-party GitHub Actions to full commit SHAs.** This is the cleanest next hardening pass and matches GitHub’s own guidance. [13](#) [14](#) [30](#)
4. **Add catalog performance and scale tests.** Create synthetic large artifact trees and assert acceptable latency/memory for `build_catalog`, `query_catalog`, `build_lineage_edges`, and explorer renders. This addresses the most obvious future M36 usability bottleneck before it becomes user-visible debt. [6](#) [7](#)
5. **Build a thin exporter from current lineage/evidence records to OpenLineage and/or W3C PROV as a proof of concept.** This gives M36 external interoperability value without changing the canonical artifact model. [3](#) [26](#)

Recommended code-level changes

The following changes are the most leverage-efficient code interventions given the current file layout and contracts. The current relevant surfaces already exist in `src.catalog`, `src.cli.query_catalog`, and `src.cli.explore_catalog_evidence`, so M36 can be evolutionary rather than disruptive. [6](#) [8](#) [9](#)

Recommended change	Primary files	Effort	Why it matters
Add optional derived local metadata index export	<code>src/catalog/indexer.py</code> , <code>src/catalog/query.py</code> , <code>src/cli/query_catalog.py</code>	Medium	Improves search/summary latency without changing source-of-truth artifacts
Add standards exporters	<code>src/catalog/lineage.py</code> , new <code>src/catalog/exporters/*.py</code>	Medium	Enables OpenLineage / PROV interoperability and future ecosystem integration
Add dataset / feature lineage fingerprints	<code>src/catalog/models.py</code> , artifact manifest producers, validation surfaces	High	Strengthens reproducibility beyond run-level artifacts into data-contract traceability
Add catalog performance benchmark tests	new <code>tests/test_catalog_performance.py</code> , fixtures under <code>tests/fixtures/</code>	Medium	Converts scale concerns into measurable, merge-gated regressions
Align release and package versioning	<code>pyproject.toml</code> , <code>.github/workflows/release.yml</code> , release notes templates	Low	Reduces confusion across tags, package metadata, and milestone docs
Harden workflow security	<code>.github/workflows/*.yml</code>	Medium	Closes a well-understood supply-chain risk surface
Improve milestone workflow reuse	<code>.github/workflows/ci.yml</code> , <code>.github/workflows/milestone_validation.yml</code>	Low to medium	Ensures milestone validation and CI cannot drift apart

Primary sources consulted and limitations

The main repository sources were the repo homepage, README, M35 release notes, M35 foundation and validation docs, `src.catalog` exports and models, `src.cli` query/explorer entry points, M35 tests, the CI/release/milestone workflows, `pyproject.toml`, `requirements-dev.lock`, and GitHub issues #394 and #401. ^{1 12 6 13 14 15 16 2 3 7 8 9 10 4 5}

The principal external sources were MLflow's tracking docs, OpenLineage docs, DVC docs, Qlib docs, W3C PROV, GitHub's GitHub Actions secure-use guidance, the public GitHub repositories for QuantConnect LEAN and NautilusTrader, and the recent provenance paper **yProv4ML**. ³³

Two limitations should stay explicit. First, I found **no direct literal “M36” spec** in the reviewed repo artifacts, so the roadmap above is an evidence-based successor-milestone proposal, not a restatement of an existing M36 document. Second, the GitHub connector surfaces issue-level numeric milestone metadata but did not provide a full milestone-title listing in the reviewed materials, so the interpretation of numeric milestone: 36 as a GitHub milestone identifier is strong but still inferential rather than directly title-confirmed.

- 1 18 31 <https://github.com/christophermoverton/stratlake-trade-engine>
<https://github.com/christophermoverton/stratlake-trade-engine>
- 2 [m35_release_notes.md](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m35_release_notes.md)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m35_release_notes.md
- 3 [m35_evidence_catalog_foundation.md](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m35_evidence_catalog_foundation.md)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m35_evidence_catalog_foundation.md
- 4 [M35: Catalog Schema and Source Expansion for Robustness-Aware Evidence](https://github.com/christophermoverton/stratlake-trade-engine/issues/394)
<https://github.com/christophermoverton/stratlake-trade-engine/issues/394>
- 5 [M35: Merge Artifact Lineage Research Catalog Branch to Main](https://github.com/christophermoverton/stratlake-trade-engine/issues/401)
<https://github.com/christophermoverton/stratlake-trade-engine/issues/401>
- 6 [__init__.py](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/__init__.py)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/__init__.py
- 7 [m35_evidence_discovery_validation.md](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m35_evidence_discovery_validation.md)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m35_evidence_discovery_validation.md
- 8 [query_catalog.py](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/query_catalog.py)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/query_catalog.py
- 9 [explore_catalog_evidence.py](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/explore_catalog_evidence.py)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/explore_catalog_evidence.py
- 10 [test_m35_evidence_discovery_validation.py](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/tests/test_m35_evidence_discovery_validation.py)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/tests/test_m35_evidence_discovery_validation.py
- 11 17 21 24 32 33 <https://mlflow.org/docs/latest/ml/tracking/>
<https://mlflow.org/docs/latest/ml/tracking/>
- 12 [README.md](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/README.md)
<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/README.md>
- 13 [ci.yml](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/ci.yml)
<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/ci.yml>
- 14 [release.yml](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/release.yml)
<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/release.yml>
- 15 [milestone_validation.yml](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/milestone_validation.yml)
https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/milestone_validation.yml
- 16 [pyproject.toml](https://github.com/christophermoverton/stratlake-trade-engine/blob/main/pyproject.toml)
<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/pyproject.toml>

19 **models.py**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/models.py>

20 **requirements-dev.lock**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/requirements-dev.lock>

22 **<https://www.w3.org/TR/prov-overview/>**

<https://www.w3.org/TR/prov-overview/>

23 28 **<https://github.com/QuantConnect/Lean>**

<https://github.com/QuantConnect/Lean>

25 **<https://dvc.org/doc>**

<https://dvc.org/doc>

26 **<https://openlineage.io/docs/>**

<https://openlineage.io/docs/>

27 **<https://qlib.readthedocs.io/en/latest/>**

<https://qlib.readthedocs.io/en/latest/>

29 **https://github.com/nautechsystems/nautilus_trader**

https://github.com/nautechsystems/nautilus_trader

30 **<https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions>**

<https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions>